

# Forma & Co. — Furniture Store

Software Engineer Intern technical assessment — submission notes

- **Candidate:** Rensith Udara Gonalagoda
- **Live site:** <https://furniturestore.rensithudara.com/>
- **Repository:** <https://github.com/RensithUdara/Furniture-Store-WebApplication>
- **Admin panel:** `/admin` on the live site

A responsive furniture store with a customer storefront and a staff admin panel. Customers browse, order and pay online through PayHere (sandbox), by cash on delivery, or by sending the order over WhatsApp. Staff manage products, stock, orders, promotions and reports.

The full reference (every feature, route and API) is in `README.md`. This document is the short version.

## 1. Technologies

| Layer            | Choice                                                                        |
|------------------|-------------------------------------------------------------------------------|
| Framework        | Next.js 16.3 (App Router, Route Handlers), React 19.3, TypeScript             |
| Database         | Supabase PostgreSQL, with Row Level Security and database functions           |
| Authentication   | Supabase Auth, cookie sessions                                                |
| File storage     | Supabase Storage (product and promo images)                                   |
| Payments         | PayHere Sandbox (hosted checkout and server notification), PayHere Refund API |
| Messaging        | WhatsApp click-to-chat order message; Resend for staff and reminder email     |
| Validation       | Zod, shared by the browser and the server                                     |
| Styling          | Hand-written responsive CSS with design tokens; Lucide icons                  |
| Documents        | pdf-lib for branded invoices; own CSV and Excel writers for exports           |
| Tooling, hosting | Prettier, Playwright, PGLite; Hostinger Node.js hosting                       |

## 2. Setup

Requires Node.js 22 or newer and a Supabase project.

1. `npm install`
2. Copy `.env.example` to `.env.local` and fill in the app address ( `NEXT_PUBLIC_APP_URL` ), the three Supabase keys, the PayHere merchant id and secret with `PAYHERE_MODE=sandbox` , and `WHATSAPP_BUSINESS_NUMBER` . Each variable is explained in that file.
3. In the Supabase SQL editor, run `supabase/migrations/001_store.sql` to `014_marketing.sql` in order, then `supabase/seed.sql` for the sample catalogue.
4. Register an account in the app, then make it an admin: `update public.profiles set role = 'ADMIN' where email = 'you@example.com';`

5. `npm run dev` and open `http://localhost:3000`. The admin panel is at `/admin`.

PayHere only accepts payments from a domain registered in its merchant portal, so online payment works on the deployed site and not on `localhost`. Cash on delivery and WhatsApp ordering work everywhere.

### 3. Architecture

One Next.js application serves the storefront, the admin panel and the API.

```
Browser → Next.js pages (server-rendered) → services/ → Supabase (PostgreSQL + RLS)
        → /api route handlers → Zod validation → database functions
PayHere → /api/payments/notification (signature verified) → record_payment()
```

- `src/app` — pages and API route handlers. `src/proxy.ts` refreshes the session and guards private routes.
- `src/services` — server-only data access, one file per area (catalogue, orders, rewards, admin, marketing).
- `src/lib` — shared logic: validation schemas, permissions, payments, rate limiting, reports, SEO.
- `src/components` — UI. Server components by default; client components only where there is interaction (cart, forms, gallery).
- Pages read data on the server. Every change goes through an API route that validates the input and then calls the database.

### 4. Database design

About 25 tables in PostgreSQL, grouped as follows.

| Area         | Tables                                                                                                                                                                                                                                 |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Catalogue    | <code>categories</code> (two levels), <code>products</code> , <code>product_variants</code> (finish, SKU, price, stock), <code>product_images</code> , <code>bundles</code> , <code>flash_sales</code>                                 |
| Orders       | <code>orders</code> , <code>order_items</code> , <code>payment_events</code> , <code>order_events</code> , <code>return_requests</code> , <code>refunds</code>                                                                         |
| Customers    | <code>profiles</code> , <code>addresses</code> , <code>wishlist_items</code> , <code>loyalty_ledger</code> , <code>product_reviews</code> , <code>saved_carts</code> , <code>stock_alerts</code> , <code>newsletter_subscribers</code> |
| Store        | <code>store_settings</code> , <code>delivery_zones</code> , <code>coupons</code> , <code>promo_slides</code>                                                                                                                           |
| Staff, audit | <code>staff_roles</code> , <code>stock_movements</code> , <code>activity_log</code> , <code>rate_limits</code>                                                                                                                         |

- **Price and stock live on the variant**, because each finish of a product is priced and stocked separately.
- **An order line stores the product name, finish and unit price as they were when ordered**, so later catalogue edits never change past orders.
- **Orders are created by one database function** (`place_order`). In a single transaction it locks the variants, checks stock, reserves it, applies sale prices, set discounts, coupons and reward points, and writes the totals.

- **Status changes, payments, refunds and stock adjustments also go through functions**, which enforce the allowed transitions.
- Schema changes are numbered migrations that are safe to run twice. The app detects a migration that has not been run and hides that feature instead of failing.

## 5. Important technical decisions

---

- **Totals are computed in the database, never trusted from the browser.** The browser sends variant ids and quantities only. What the customer sees in the bag is an estimate; the saved order is the authority.
- **Stock is reserved when the order is saved, not when it is paid.** Two customers cannot buy the last item. Unpaid PayHere orders are cancelled automatically after a set time and their stock is returned.
- **An order is marked paid only by PayHere's server notification**, after its MD5 signature, merchant id and amount are checked. Returning to the site from the payment page does not mark anything paid. Duplicate notifications are ignored.
- **WhatsApp orders are saved first**, then WhatsApp opens with a prepared message containing the items, total and order number, so the store has a record even if the message is never sent.
- **Supabase instead of a custom backend.** Auth, database and storage come from one service, and Row Level Security gives a second layer of access control below the API.
- **No CSS or UI framework.** A small token-based stylesheet keeps the build simple and the design consistent on phone, tablet and desktop.
- **Idempotent checkout.** Each attempt carries a key, so a double click or a retry returns the same order instead of creating two.

## 6. Security approach

---

- **Authentication:** Supabase Auth with HTTP-only cookie sessions and email confirmation. Passwords are never handled or stored by the app.
- **Authorisation, checked three times:** the page guard, the API route ( `requirePermission` ), and the database (Row Level Security policies and checks inside each function). Hiding a menu item is never the protection.
- **Roles:** `CUSTOMER` , `STAFF` with a configurable set of admin areas, and `ADMIN` . Customers can read only their own orders, addresses and points.
- **Input validation:** every API body is parsed with Zod, and the database repeats the important checks with constraints.
- **Secrets:** the Supabase secret key and PayHere secrets are server-only environment variables and are never sent to the browser. The secret key is used only where a signed-in user does not exist (payment notification, guest orders, staff creation).
- **Request forgery:** every state-changing API call checks the request origin.
- **Rate limiting:** sign-in, registration, checkout, coupon checks, order lookup and newsletter signup are limited per account or address, with counts kept in the database.
- **Guest orders** are reached only through an unguessable token, and guest order pages are kept out of search engines.
- **Uploads** are restricted by file type and size, and to staff with the matching permission.
- **Headers:** `X-Frame-Options: DENY` , `X-Content-Type-Options: nosniff` , a strict referrer policy.
- **Audit trail:** an activity log of staff changes and a stock history, both written by database triggers.